

UNIVERSIDAD CATÓLICA DE MURCIA (UCAM)

2ª Ed. Máster en IA Aplicada a la Ciberseguridad

Curso académico: 14/10/2025 – 31/08/2026

Detección de secretos en código y pipelines mediante reglas y LLM

*Combinación de expresiones regulares y entropía con un modelo de lenguaje para
reducir falsos positivos, integración en pre-commit y CI/CD, medición de resultados
y políticas de rotación y ocultación*

Trabajo Fin de Máster — Documento técnico

Autor/a: Ángela Ramírez Barajas

Director: Juanjo Salvador

Repositorio: <https://github.com/aramirezbarajas/secret-scan-tfm>

Resumen ejecutivo

Las credenciales (claves API, tokens, contraseñas, PEM) se filtran con frecuencia en repositorios Git. Los escáneres basados en reglas —expresiones regulares y umbrales de entropía— son rápidos e integrables en pre-commit y CI, pero generan un volumen alto de **falsos positivos**: un token de test, un mock o un ejemplo en documentación dispara la misma alerta que un secreto de producción. Ese ruido dificulta la adopción real de la herramienta.

La solución es un **pipeline híbrido**. Gitleaks (v8.30.1) detecta candidatos. Un LLM local (Llama 3.1 8B, vía Ollama, sin fine-tuning ni API cloud) no busca secretos: **tria únicamente los falsos positivos**. Los verdaderos positivos no se reclasifican, de modo que un error del modelo no puede silenciar una credencial real. En el flujo de desarrollo, Gitleaks bloquea en pre-commit y GitHub Actions; el LLM queda para triaje en lote por latencia (5–15 s por candidato).

Aportaciones: (i) un filtro contextual medible sobre Gitleaks y CredData, sin entrenamiento; (ii) evidencia de que el diseño del prompt —no el tamaño del modelo— determina el acierto; (iii) una separación operativa reglas/LLM integrable; (iv) políticas de rotación y ocultación para código y pipelines.

Evaluación sobre CredData (Samsung): Gitleaks obtiene 85,85 % de precisión y 45,86 % de recall (F1 59,79 %) sobre 8.210 hallazgos (6.845 verdaderos positivos, 1.128 falsos positivos). Con un prompt genérico el LLM acierta el 34 % de una muestra de 200 FP; con un prompt de contexto (rutas de test, mocks, documentación) acierta el **99 %**, con cero regresiones. La precisión híbrida proyectada es 99,97 %; el recall no cambia. No se reclama un F1 superior a sistemas con LoRA (Rahman et al., 2025): la tarea es el triaje de alertas, no la detección global. El código, con README de reproducción, está en <https://github.com/aramirezbarajas/secret-scan-tfm>.

Palabras clave: detección de secretos, Gitleaks, LLM, falsos positivos, DevSecOps, CredData.

1. Introducción

1.1 El problema y su relevancia

Un secreto en código es un dato que no debería ser público: una clave de cloud, un token de API, una contraseña o una clave privada. Si queda en Git, cualquiera con acceso al historial puede usarlo. El volumen del problema es conocido: millones de secretos aparecen cada año en repositorios públicos (Basak et al., 2023).

La prevención —gestores de secretos, variables de entorno, no hardcodear— no basta. OWASP recomienda complementar con detección automática *shift-left*: un hook **pre-commit** que intercepte el secreto antes del push y un escaneo en **CI/CD** como segunda barrera (OWASP, 2024a, 2024b). El cuello de botella no es tanto encontrar formatos conocidos como **convivir con las alertas**. Un escáner que pita por cada `MOCK_ACCESS_TOKEN` en `tests/fixtures` acaba desactivado o ignorado.

Este trabajo se plantea, por tanto, un problema de **adopción**: reducir el ruido de un detector de reglas ya usable, sin perder los secretos que ese detector sí encuentra, e integrar el conjunto en el ciclo Git.

1.2 Soluciones existentes

Los escáneres de reglas combinan patrones (regex para claves AWS, JWT, PEM, tokens GitHub) y **entropía** para cadenas muy aleatorias (Meli et al., 2019).

Gitleaks es representativo: rápido, offline e integrable en pre-commit y GitHub Actions (Zanev, 2024). En el benchmark oficial de CredData (Samsung, 2022) obtuvo 52,6 % de precisión y 24,4 % de recall (Yun et al., 2021). Versiones posteriores, como la usada aquí (8.30.1), mejoran la precisión pero no el hecho de tratar cada coincidencia de forma aislada. **detect-secrets** añade una *baseline* de falsos positivos conocidos, a costa de mantenimiento manual (Yelp, 2024). **TruffleHog** verifica si la credencial sigue viva contra APIs: más confianza cuando hay red, más lento y éticamente delicado sobre secretos ajenos (Truffle Security, 2024).

La evaluación rigurosa exige ground truth. **CredData** etiqueta líneas de repositorios públicos como secreto real (T), falso positivo (F) o no aplicable (X) (Yun et al., 2021). **SecretBench** es mayor y reside en BigQuery (Basak et al., 2023). Aquí se usa CredData por ser reproducible en local y porque incluye Gitleaks en su comparativa.

Sobre esa base hay tres líneas de aprendizaje automático. **CredSweeper** entrena un modelo clásico y obtiene el mejor F1 oficial en CredData (0,859), con el coste de mantener el modelo (Samsung, 2022). **Biringa y Kul (2025)** usan embeddings y un clasificador sobre CredData (F1 0,973). **Rahman et al. (2025)** extraen candidatos por regex y clasifican con LLM sobre SecretBench; con Llama 3.1 8B ajustado (LoRA) llegan a F1 0,985. Es el trabajo más cercano: usan el mismo tamaño de modelo. Difieren en el objetivo (F1 global de un clasificador frente a triaje de FP de Gitleaks), en el corpus y en el requisito de entrenamiento.

La laguna no es «usar un LLM para secretos», ya demostrado. Es cuantificar qué aporta un **filtro contextual sin fine-tuning** sobre un escáner que un equipo puede desplegar mañana, y cerrar el ciclo con pre-commit, CI y una política de respuesta.

2. Aportaciones

Tras este trabajo queda, de forma concreta:

1. Un pipeline híbrido **reglas + LLM local** en el que Gitleaks detecta y Llama 3.1 8B (Ollama) filtra falsos positivos, sin reentrenamiento ni servicios cloud.
2. Una métrica operativa de **precisión híbrida** que conserva el recall de las reglas, junto con una comparativa controlada de dos prompts (genérico vs. contextual) sobre los mismos candidatos de CredData.
3. Una separación de responsabilidades **desplegable**: Gitleaks en pre-commit y GitHub Actions; LLM en lote.
4. Un marco breve de **rotación y ocultación** para secretos en código y en pipelines (referencias a secretos del proveedor, no valores en el YAML).
5. Un repositorio con README que permite reproducir el flujo experimental y el demo DevSecOps: <https://github.com/aramirezbarajas/secret-scan-tfm>.

3. Definición

3.1 Bases y arquitectura

La solución se construye sobre dos ideas. Primera: las reglas son insustituibles para la **detección** en un hook de Git (latencia subsegundo, sin GPU). Segunda: un modelo de lenguaje puede usar **contexto** (ruta, nombre de variable, fragmento de código) que las reglas ignoran.

El pipeline tiene tres pasos (Figura 1). Gitleaks escanea el corpus y emite hallazgos. Cada hallazgo se cruza con las etiquetas de CredData: verdaderos positivos (alerta y T), falsos positivos (alerta y F/X) y no detectados (T sin alerta). El LLM recibe solo la rama de FP, con ruta, regla, coincidencia y un recorte de código. Responde si el hallazgo es un secreto real o un falso positivo. Si declara FP, la alerta se descarta para el recuento de precisión híbrida.

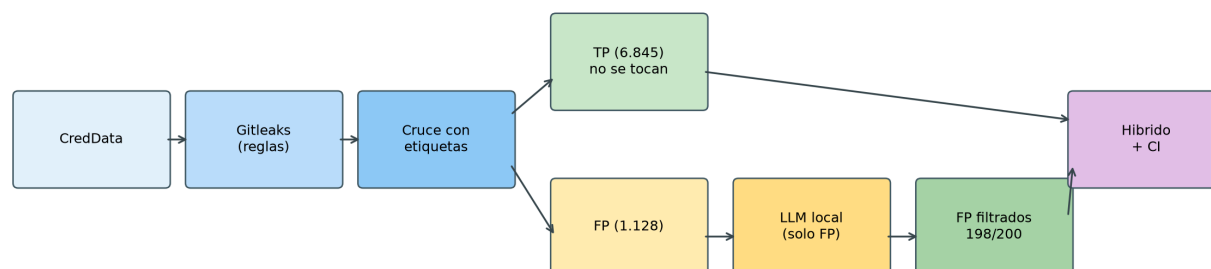


Figura 1. Pipeline híbrido. Las reglas detectan; el LLM solo reclasifica falsos positivos.

La **precisión híbrida** se define como $TP / (TP + FP \text{ restantes})$, con los TP fijos. Es una proyección cuando el LLM no procesa todos los FP: se asume que la tasa de acierto de la muestra se sostiene. Esa hipótesis se discute en la evaluación.

3.2 Decisiones de diseño

El modelo no extrae candidatos. Si el LLM buscara secretos por su cuenta, competiría con Gitleaks en recall y exigiría recorrerlo todo el repositorio a coste de segundos por archivo. Aquí las reglas cubren cobertura y velocidad; el modelo cubre discriminación.

Los verdaderos positivos no se reclasifican. El riesgo de un filtro semántico es silenciar un secreto real. Al no enviar TP al modelo, el recall del híbrido es exactamente el de Gitleaks. La métrica del LLM es la tasa de FP bien descartados, no un F1 de detector autónomo. Esta decisión alinea el sistema con el enunciado (reducir falsos positivos) y con un uso seguro en CI: lo que las reglas marcaron como T no se apaga por un prompt.

Inferencia con prompt, no fine-tuning. Rahman et al. muestran que LoRA supera al prompting en F1 global. Aquí no hay GPU de entrenamiento ni corpus de fine-tuning. El mismo Llama 3.1 8B, en local, permite iterar prompts en días. La apuesta es que, para la subtask acotada de reconocer tests, mocks y documentación, reglas explícitas en el prompt bastan.

LLM fuera del commit. Cinco a quince segundos por candidato son incompatibles con un hook diario. Gitleaks en pre-commit y en GitHub Actions aporta el bloqueo; el LLM se reserva a análisis en lote o a un runner con Ollama.

Diferencia respecto al estado del arte. Frente a Gitleaks, el híbrido añade contexto. Frente a Rahman et al. o Biringa y Kul, no se reclama ganar en F1: ellos entrenan un clasificador sobre todos los candidatos; este trabajo filtra ruido de un escáner ya integrable, sin entrenamiento.

3.3 Integración y políticas

En desarrollo, Gitleaks se ejecuta antes de cada commit y de nuevo en cada push y pull request (historial Git completo). Los secretos de pipeline no se escriben en el YAML: se referencian como secretos del proveedor. Un hallazgo confirmado en `main` implica rotar en menos de 24 h; en un PR, bloquear el merge. Las excepciones de allowlist requieren justificación. La evidencia de bloqueo local y de CI en verde se recoge en el Anexo B.

4. Desarrollo

La construcción no partió de un clasificador entrenado, sino de medir Gitleaks sobre CredData y añadir el filtro. El primer obstáculo fue operativo: el corpus (~1 GB) y el informe JSON no caben en el repositorio; se generan en local (Anexo A). El cruce con meta/ exigió alinear rutas y números de línea; sin ese cruce no hay TP/FP, solo “leaks”.

El segundo obstáculo fue el **prompt**. La versión genérica («¿secreto real o falso positivo?») se apoyaba en el formato del valor: un PEM o un token con pinta de clave se clasificaban como auténticos aunque vivieran en tests. Sobre 200 FP el acierto fue del 34 %. Se descartó como baseline insuficiente, no como fallo del modelo: el mismo Llama 3.1 8B, temperatura 0, se reutilizó.

La versión contextual añade reglas de ruta (`/test/`, `/fixtures/`, `/docs/`, `/mock/`), nombres (MOCK, placeholder) y la instrucción de que un formato creíble no basta. El salto a 99 % de acierto, con **cero empeoramientos** respecto a v1, confirma que el desarrollo crítico fue catalogar patrones de FP, no cambiar de arquitectura.

Un caso residual persistió: un PEM de ejemplo embebido en documentación *inline* dentro de una ruta de aplicación, sin segmentos de test. El formato ganó a señales débiles (IDs 123123, clave truncada). No se implementó una tercera versión de prompt: el plazo del TFM y el rendimiento ya alto de v2 lo desaconsejaron.

El tercer obstáculo fue la **latencia**. Procesar 1.128 FP a 5–15 s implicaría horas. Se acotó la evaluación LLM a N = 200, un alcance realista (criterio de ajuste alcance–resultado de la guía) a costa de proyectar el resto. El LLM no se colocó en GitHub-hosted runners: dependen de Ollama.

La implementación detallada (scripts, configuración Gitleaks, demo-repo) vive en el repositorio, no en este documento.

5. Evaluación

Protocolo. CredData generado en local: 66.898 líneas etiquetadas, 15.104 secretos reales (T), 11.393 archivos. Gitleaks 8.30.1; LLM llama3.1:8b, temperatura 0. Métricas: TP = alerta y T; FP = alerta y

F/X; FN = T sin alerta; precisión = $TP/(TP+FP)$; recall = T detectados / total T; tasa de filtrado LLM = FP bien descartados / FP evaluados. Las salidas de consola están en el Anexo A.

Baseline (solo Gitleaks). 8.210 hallazgos en 56,8 s sobre ~1,02 GB. Sobre líneas etiquetadas: 6.845 TP, 1.128 FP, precisión **85,85 %**, recall **45,86 %**, F1 **59,79 %**. Gitleaks es útil como primera red (la mayoría de alertas etiquetadas son secretos reales) y deja 8.177 FN: formatos que las reglas por defecto no cubren (credenciales en URL, nonces, secretos propietarios). El híbrido no pretende cerrar esos FN. El problema que sí ataca son 1.128 FP, un volumen de triaje inviable en un hook diario.

Filtro LLM (N = 200 FP). Prompt v1: 68/200 (34 %), precisión híbrida proyectada 98,11 %. Prompt v2: 198/200 (**99 %**), 99,97 % proyectada. Sobre 199 candidatos comunes: 130 correcciones v1→v2, 0 empeoramientos, 1 fallo persistente (el PEM *inline*). La Figura 2 muestra el salto de precisión; la Figura 3, que el recall no se mueve: evidencia de que el LLM es un filtro, no un detector adicional.

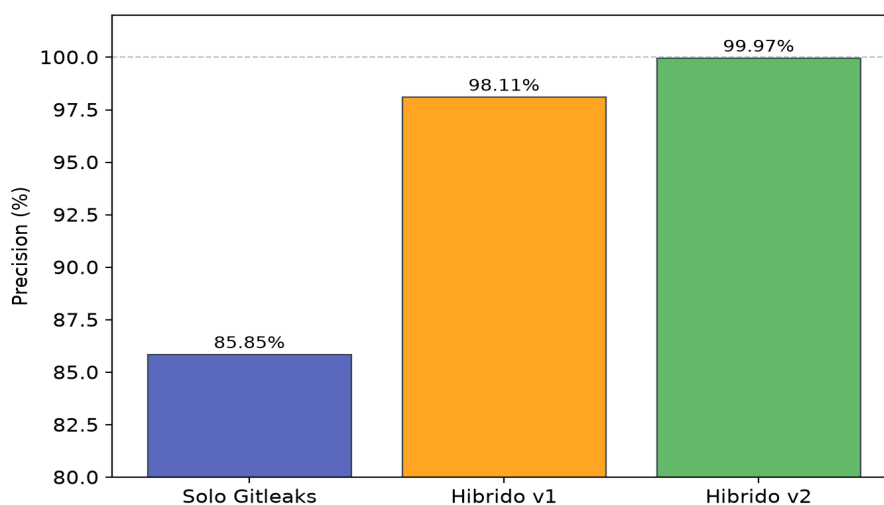


Figura 2. Precisión: reglas solas frente a reglas + LLM (v1 genérico, v2 contextual). El valor de v2 es una proyección a partir de la muestra de 200 FP.

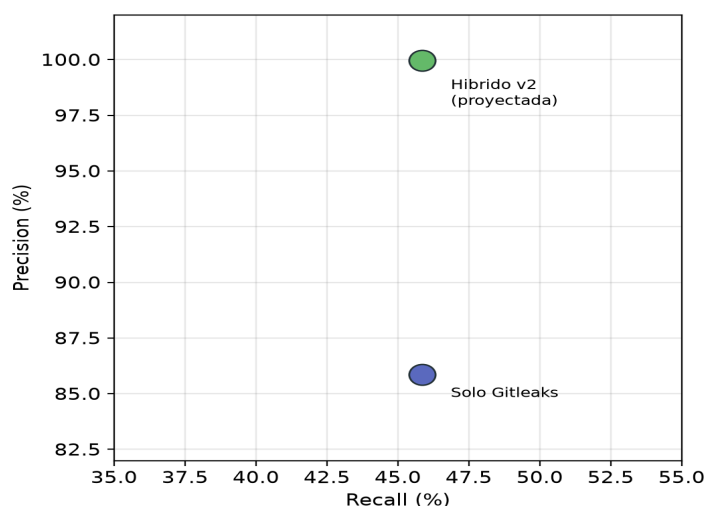


Figura 3. Precisión frente a recall. El punto híbrido permanece en el mismo recall (45,86 %).

Comparación con el estado del arte. La comparación directa y honesta es contra **Gitleaks**, la solución de reglas referenciada en la introducción y sobre la que se construye el filtro: misma herramienta, mismo corpus, mismos TP, menos FP. No hay comparación numérica directa con Rahman et al. (2025): usan SecretBench, extraen con regex propio y maximizan F1 de un clasificador con LoRA. Mezclar esas cifras con la precisión híbrida de este TFM sería inflar el resultado. El valor

demostrable es operativo: 34 % → 99 % de acierto en FP de Gitleaks cambiando solo el prompt, con el mismo 8B local.

La proyección 99,97 % asume que los 928 FP no muestreados se comportan como los 200. Es una **cota alta**. Los FP de CredData se concentran en tests, mocks y documentación —justo lo que v2 codifica—, pero un muestreo estratificado o el corpus completo podría bajar el porcentaje. La hipótesis de reducción de FP ≥ 30 % queda confirmada en la muestra (99 %).

Las cifras del LLM asumen que `fp_candidates.json` no ha sido manipulado. Como control de integridad (experimento P2, no parte del detector), se simuló ataques de envenenamiento sobre ese fichero: el recuento no basta frente a etiquetas volteadas; hash y deriva de etiquetas sí detectan a partir del 5 % de veneno (Anexo C).

6. Conclusión

La expectativa inicial era que un LLM local, añadido a Gitleaks, reduciría de forma medible el ruido sin reentrenar. Eso se cumple en la muestra. Lo que no se esperaba, y es el aprendizaje principal, es **cuánto importa el prompt**: el mismo modelo pasa de inútil (34 %) a operativo (99 %) cuando el contexto de test/mock/docs se hace explícito. Añadir un LLM no es, por sí solo, la aportación.

El mayor reto no fue el modelo, sino acotar el alcance: no se procesaron los 1.128 FP, no se mejoró el recall de las reglas y no se verificó si las credenciales siguen vivas. Esas limitaciones son condiciones de validez: los resultados hablan de triaje de FP de Gitleaks en CredData, con Llama 3.1 8B y un prompt alineado a patrones repetitivos de ese corpus. Pueden no generalizar a otros lenguajes, a secretos en rutas de aplicación sin señales de test, o a otro modelo.

Con más tiempo, el trabajo continuaría por tres vías: muestreo estratificado o corpus FP completo; reglas para documentación *inline*; y, si hubiera GPU, un clasificador ligero al estilo de la literatura, midiendo no solo F1 sino latencia en CI. El detector del día a día seguiría siendo el de reglas.

Bibliografia

- Basak, S. K., Neil, L., Reaves, B., y Williams, L. (2023). SecretBench: A dataset of software secrets. En *Proceedings of the 20th International Conference on Mining Software Repositories (MSR)* (pp. 347-351). IEEE. <https://doi.org/10.1109/MSR59073.2023.00053>
- Biringa, C., y Kul, G. (2025). Detecting hard-coded credentials in software repositories via LLMs. *arXiv preprint arXiv:2506.13090*. <https://arxiv.org/abs/2506.13090>
- Meli, M., McNee, J., y Neville-Neil, G. (2019). Source code credentials hygiene: A case study. En *Proceedings of the 44th IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*. IEEE.
- OWASP Foundation. (2024a). CI/CD security cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/CI_CD_Security_Cheat_Sheet.html
- OWASP Foundation. (2024b). Secrets management cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
- Rahman, M. N., Ahmed, S. I., Wahab, Z., Sohan, S. M., y Shahriyar, R. (2025). Secret breach detection in source code with large language models. *arXiv preprint arXiv:2504.18784*. <https://arxiv.org/abs/2504.18784>
- Samsung. (2022). CredSweeper: ML-driven credential scanner. <https://github.com/Samsung/CredSweeper>
- Truffle Security. (2024). TruffleHog: Find and verify leaked credentials. <https://github.com/trufflesecurity/trufflehog>
- Yelp. (2024). detect-secrets: Tool for detecting secrets in the codebase. <https://github.com/Yelp/detect-secrets>
- Yun, J., Choi, S., Lee, Y., Sokol, O., Shim, W., Melkonyan, A., y Kuzmenko, D. (2021). CredData: A dataset of credentials for research. Samsung. <https://github.com/Samsung/CredData>
- Zanev, Z. (2024). Gitleaks. <https://github.com/gitleaks/gitleaks>

Anexos

Los anexos recogen evidencia de ejecución. No sustituyen al cuerpo: se referencian desde las secciones 3, 4 y 5.

Anexo A – Salidas de evaluación

A.1 Gitleaks sobre CredData. Escaneo de ~1,01 GB en 59,5 s; informe JSON de 8.210 hallazgos. Volcado de consola en docs/anexos/C01_gitleaks_credata.txt (*leaks found: 8083* en el log; el cruce usa el informe completo).

A.2 Cruce con etiquetas. Precisión 85,85 %, recall 45,86 %, F1 59,79 %, 6.845 TP, 1.128 FP. Salida completa en docs/anexos/C02 evaluate gitleaks.txt.

A.3 Comparativa de prompts. 199 candidatos comunes; 130 correcciones v1→v2; 0 empeoramientos; v2 198/200. Salida en docs/anexos/C03 compare llm runs.txt.

Anexo B — Pre-commit y CI

Figura B.1. Intento de commit con un secreto de demo: Gitleaks aborta el commit (*Failed, leaks found: 2*).

```
git add leaks/demo-block-me.env
git commit -m "test leak"
warning: in the working copy of 'leaks/demo-block-me.env', LF will be replaced by CRLF the next time Git touches it
Detect hardcoded secrets..... Failed
- hook id: gitLeaks
- exit code: 1

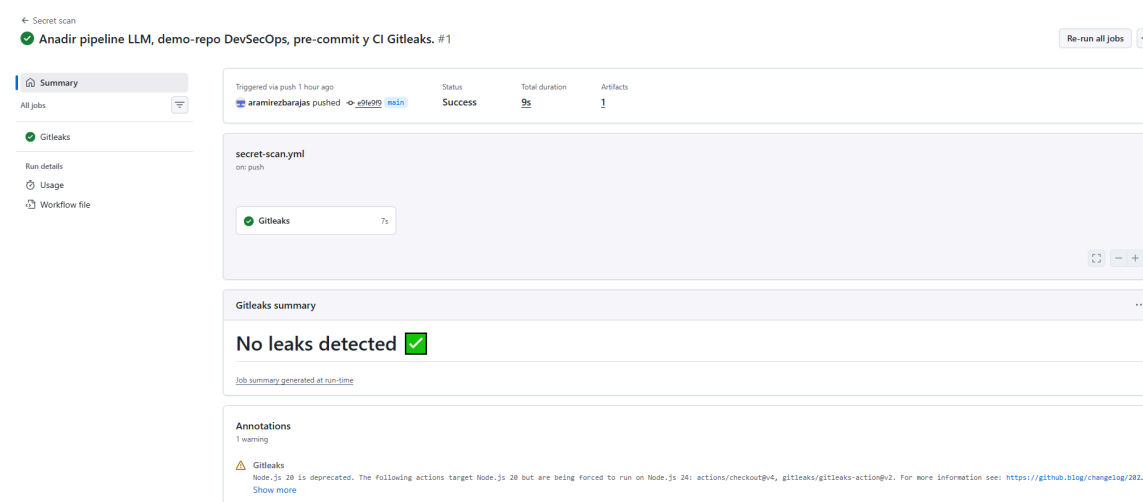
o
├── o
│   └── gitleaks
○
├── ○
└── ○
```

```
Finding: DEMO_GITLAB_TOKEN=REDACTED
Secret: REDACTED
RuleID: generic-api-key
Entropy: 4.220129
File: leaks/demo-block-me.env
Line: 5
Fingerprint: leaks/demo-block-me.env:generic-api-key:5

Finding: DEMO_GITLAB_TOKEN=REDACTED9b2d8a1f4e7c3b9d2c
Secret: REDACTED
RuleID: gitlab-pat
Entropy: 4.286790
File: leaks/demo-block-me.env
Line: 5
Fingerprint: leaks/demo-block-me.env:gitlab-pat:5

9:00AM INF 1 commits scanned.
9:00AM INF scanned ~63 bytes (63 bytes) in 204ms
9:00AM WRN leaks found: 2
```

Figura B.2. Workflow secret-scan.yml en GitHub Actions: job Gitleaks en verde, No leaks detected.



Anexo C – Integridad del dataset de evaluación (P2)

El LLM se evalúa sobre `fp_candidates.json` (1.128 FP). Un atacante que altere etiquetas falsearía las métricas sin tocar el modelo. Se simularon dos ataques en laboratorio: **inyectar filas** (añadir T en el cubo FP) y **voltear etiquetas** (F/X $\rightarrow$ T, mismo número de filas). Detectores: hash del fichero (D1), subconjunto dorado de 7 claves (D2), recuento (D3), deriva PSI de etiquetas (D4, umbral 0,2).

Ataque	Veneno	Detectado	Nota
Control	0 %	No	Fichero limpio
Injectar filas	5 % (+56)	Sí	D1, D3 y D4 alertan
Voltear etiquetas	5 % (56)	Sí	D3 no alerta; D1 y D4 sí
Voltear etiquetas	30 %	Sí	También D2 (golden)
Voltear etiquetas	1 % (11)	Parcial	D4 bajo umbral; D1 sí

Voltear etiquetas es más sigiloso: el volumen no cambia. A partir del 5 % bastan hash y PSI. Con el 1 %, hace falta un manifiesto previo. Este control no forma parte del pipeline de detección de secretos; protege la fiabilidad de la evaluación. Reproducción: `docs/POISON_P2_RESULTADOS.md` y scripts `inject_data_poison.py` / `detect_data_poison.py`.